

wkgtk-html2pdf technical manual

Typesetting with HTML

- Document development

- Using the C++ API

rev V0.1.09.20260312_01

Index

1. INTRODUCTION

Introduction	1.0
Reading this manual	1.1
Architectural Philosophy - Built for Stability	1.2

2. COMMAND LINE INTERFACE:

Basic usage	2.1
Command line options	2.2

3. OPTIMISING HTML:

Key principles	3.1
Stylesheet utilisation	3.1.1
Page declaration	3.1.2
Overflow monitoring utility	3.1.3
Units	3.1.4
Avoid Common Issues	3.1.5
Best Practices	3.1.6
Quickstart example	3.2

4. LINKS/ANCHORS:

Classic anchors	4.1
Enhanced anchors	4.2
Sidebar Indexing	4.3
Indexing Criteria	4.3.1
Enhanced vs. Classic - The Clickable Area	4.3.2
Index page	4.4

5. ARTEFACTS:

Images	5.1
External Links	5.2
Vector Graphics	5.3
Layering & Compositing	5.3.1

6. PAGE DESIGN HELPER:

Style sheets	6.1
Customising Templates	6.1.1
Customising The Page Size	6.1.1.1
Typography/Raster Synchronising	6.1.1.2
Element Calculations	6.1.1.3
Using page templates	6.1.2
Layout Containers	6.1.3
Block Layout	6.1.3.1
Flex Layout	6.1.3.2
Grid Layout	6.1.3.3
Fieldsets	6.1.3.4
Inline Containers	6.1.3.4
JavaScript helper utility	6.2
Monitored Issues	6.2.1
Font Management	6.2.2
Strategic Checklist for Template Designers	6.2.2.1
How to Resolve Overflows	6.2.2.2
Using Google Fonts and CDN's	6.2.3

7. HINTS AND TIPS:

Page Numbering	7.1
Fomattting hyperlinks for PDF	7.2
Removing the page margin for viewing	7.3
The Reference Baseline Template	7.4
Provisions	7.4.1
Note Box	7.4.1.1
Important Box	7.4.1.2
Warning Box	7.4.1.3
Index	7.4.1.4
Development Environment Setup	7.5
Pulsar Edit	7.5.1
Pulsar Edit Snippets	7.5.1.1
VSCode Snippets	7.5.1.2
Vim and Browser	7.5.2

8. CALIBRATION:

Why calibrated	8.1
The Dual-Output Strategy	8.1.1
When to Calibrate	8.1.2
When to Calibrate	8.2
Creep Validation	8.2.1
Testing Browser Calibration	8.2.2
Testing Printer Calibration	8.2.3
Quality Control	8.3
Structural Integrity	8.3.1
Engine Integrity	8.3.2
Index Integrity	8.3.3

9. C++ API:

Key Concepts	9.1
The Binary Wall (Pimpl)	9.1.1
Thread Safety & Initialization	9.1.2
The html_tree Class (DOM Construction)	9.2
The PDFprinter Class (Rendering)	9.3
Working with BLOBs	9.4
Variadic Helpers	9.5
Indexing Configuration	9.6
The index_mode Enum	9.6.1
API Implementation	9.6.2
Strategic Usage	9.6.3
GTK Development	9.7
Headless applications	9.7.1
GTK Applications	9.7.2
Headless Daemons	9.7.3
Service development	9.7.3.1

1. Introduction

wkgtk-html2pdf is a powerful and easy-to-use API for converting HTML content into high-quality PDF documents using WebKitGTK. Developed as a modern alternative to legacy tools like wkhtmltopdf—which has been archived since 2023—this project fills the gap for reliable, up-to-date HTML-to-PDF conversion. It provides a clean, intuitive C++ API that simplifies HTML generation by eliminating the need for string literals and explicit closing tags, making it ideal for embedding in applications. The tool supports advanced features such as internal anchors, links, and nested sidebar indexing, enabling the creation of structured, navigable PDFs. A simple command-line interface is also available for quick, one-off conversions. With its focus on maintainability and modern development practices, wkgtk-html2pdf is designed to be a reliable choice for developers seeking a dependable solution for PDF generation.

This project is entirely inspired by wkhtmltopdf as when we were searching for a replacement we simply could not find one, and even the paid for options seemed overly complex where they needn't be and severely lacking where they should. We have tried to iron out some of the issues we have encountered along the way and have produced this manual to assist with those unavoidable issues that must be worked around. We hope that you find this project useful and we would be very much appreciative of any contribution, be that funding, documentation, testing, or design.

wkgtk-html2pdf includes a set of built-in, pre-configured CSS templates for all standard ISO and US paper sizes, ensuring compatibility and consistency across different regions and use cases. These templates are designed to prevent the generation of extraneous blank pages by precisely defining page dimensions, margins, and layout constraints using CSS custom properties.

Each template uses a clean, modular structure with variables for page width, height, and margin, allowing for easy customization while maintaining reliable output. A lightweight JavaScript utility is included to monitor content overflow in real time—when content exceeds the available space, the subpage border turns red; when it fits perfectly, the border turns green. This visual feedback helps you quickly identify and resolve layout issues before conversion.

The templates also include print-specific CSS rules that ensure clean, artefact-free output when the HTML is rendered to PDF, while maintaining visual guides during development for easier debugging and refinement.

In **wkgtk-html2pdf** we have endeavoured to create a system that does not require you to learn additional languages; all you should need is a reasonable understanding of HTML.

1.1 Reading this manual

This manual is a self fulfilling prophecy as it is developed entirely using wkgtk-html2pdf so the layout the code that is used to write it is in itself a tutorial. If you are uncertain about how something is done then is worth reviewing the HTML and CSS used to generate it.

Note: The strict guidance in this manual is intended for the generation of high quality reliable screen to print multi page technical manuals and other mission critical documents. For non critical documents and forms it may not be necessary to adhere to certain aspects.

1.2 Architectural Philosophy: Built for Stability

Unlike many wrapper libraries that expose internal dependencies to the host application, wkgtk-html2pdf is engineered with a strict Binary Interface (ABI). We utilize the Pimpl (Pointer to Implementation) idiom throughout the C++ API to create a "firewall" between the library's internal logic and your application. This tedious but essential design choice provides several critical benefits:

Zero Dependency Leakage - Your project does not need to link against, or even be aware of, the specific versions of WebKitGTK, PoDoFo, or GLib used internally.

Upgrade Without Recompile - You can swap the underlying library (.so or .dll) for a newer version containing security patches or performance internal improvements without ever needing to recompile your main application.

Sovereign Memory Management - By avoiding C++ Smart Pointers in the public headers, which often cause ABI breakage across different compiler versions, we use raw-pointer handles and explicit cleanup functions (**PDF_FreeHTML**, **PDF_FreeBlob**). This ensures the library remains stable across different development environments and toolchains.

2. Command line interface (CLI)

The CLI is the simplest way to get started with wkgtk-html2pdf. It allows you to convert HTML files into PDFs with minimal setup.

2.1 Basic Usage

To generate a PDF using the default settings use the following command:

```
html2pdf -i input.html -o output.pdf
```

Generate a basic pdf with default settings (A4 portrait).

2.2 Command line options

The following options are available for the cli.

Option	Description
-h, --help	Display the help message with all available options.
-v, --verbose	Set the log level (1-7), with higher values providing more detailed output. Logs are written to the system journal.
-i, --infile	Specify the source HTML file to convert.
-o, --outfile	Specify the output PDF file name.
-O, --orientation	Set page orientation: portrait or landscape.
-s, --size	Set the page size (Default A4)
	- ISO (A): A0, A1, A2, A3, A4, A5, A6, A7, A8, A9, A10
	- ISO (B): B0, B1, B2, B3, B4, B5, B6, B7, B8, B9, B10
	- ISO (C): C0, C1, C2, C3, C4, C5, C6, C7, C8, C9, C10
	- US: Letter, Legal, Tabloid
	- ANSI: ANSIA, ANSIB, ANSIC, ANSID, ANSIE
	- Architectural: ArchA, ArchB, ArchC, ArchD, ArchE
	- Other: SRA0, SRA1, SRA2, SRA3, SRA4
--index	Create anchor points for indexing: classic or enhanced for nested sidebar indexing.
-r --relative-uri	look for resources such as images in, or relative to, the current working folder.

3. Optimising HTML

To generate a clean, professional PDF, you must optimize your HTML to ensure it aligns with the desired page size and layout. The output is entirely governed by the CSS and HTML structure, so the quality of the final PDF depends on how well your content is designed.

3.1 Key Principles

3.1.1 Use the correct stylesheet

1. Each stylesheet is named according to the page size and orientation it supports (e.g. A4-portrait.css, ANSI-A-landscape.css).
2. Link to the appropriate stylesheet in your HTML:

Example:

```
<link rel="stylesheet" href="/usr/share/wk2gtkpdf/A4-portrait.css">
```

Applying the Style Sheet for A4 portrait.

Note: Bundled templates are legacy. For current, customizable templates, use the automated tool at inplico.uk (free with attribute)

3.1.2 Use the correct classes

1. Use the **.page** class to define the overall page
2. Use the **.subpage** class to define content areas.

Example:

```
<div class="page">
  <div class="subpage">
    <!-- Your page content here -->
  </div>
</div>
```

Initialising page boundaries

3.1.3 Monitor the overflow

1. If you are not using our linter then Include the JavaScript utility to monitor content overflow.
2. If overflow is detected the margin turns red.

This script provides real-time feedback, helping you identify and fix layout issues before conversion ([See Section 6.2](#)).

```
<script src="/usr/share/wk2gtkpdf/overflow-monitor.js"></script>
```

Declaring the JavaScript to monitor the overflow.

3.1.4 Units: Points vs. Pixels

While web design traditionally relies on pixels (**px**), high-stakes typesetting and desktop publishing require points (**pt**).

The primary reason for this distinction is sub-pixel rounding. Because a physical screen cannot render "half a pixel," the engine must round fractional values up or down. In standard HTML-to-PDF generators, these tiny rounding errors accumulate over a large document, causing significant "creep" where content shifts further out of alignment with every passing page.

Our bundled templates are specifically engineered to isolate these rounding issues to the individual page level, preventing cumulative drift. However, to achieve a document that is truly pixel-perfect and indistinguishable between the Linter Viewer and the final PDF, you should always define measurements in points. Using points bypasses the browser's display scaling and ensures your layout remains 1:1 with the physical print dimensions.

The Conversion Rule: To convert your existing web measurements to the required print standard, simply multiply any pixel value by 0.75 (e.g., $16\text{px} * 0.75 = 12\text{pt}$).

Note: While you can manually convert pixels to points using the 0.75 multiplier, our commercial **Refinery Linter** can automate this process across entire stylesheets.

3.1.5 Avoid common issues

By designing your HTML and CSS carefully, you have full control over the final PDF output. Avoid common issues: Mismatched stylesheets, incorrect classes, or content overflow can lead to blank pages, cut-off content, or incorrect scaling.

The overflow detection script helps you identify and fix layout issues before conversion.

3.1.6 Best Practices

1. **Always test the layout** - Use the overflow detection script to ensure content fits within the page. Adjust the CSS or content as needed to avoid overflow.
2. **Use the correct stylesheet** - Ensure the stylesheet matches the CLI arguments (page size and orientation).
3. **Keep it consistent** - Use the same page size and orientation in both the CSS and CLI arguments.

3.2 Quick start - A4 Portrait Layout

Below is a minimal, single-page example utilizing the built-in overflow monitor and missing-font detector.

Example:

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="/usr/share/wk2gtkpdf/A4-portrait.css">
  </head>
  <body>
    <div class="page">
      <div class="subpage">
        <h1>My Document</h1>
        <p>This content will be rendered in A4 portrait format.</p>
      </div>
    </div>
  </body>
  <script src="/usr/share/wk2gtkpdf/overflow-monitor.js"></script>
</html>
```

Minimal single page example.

Each declaration of a **.page** and **.subpage** container will create a new page. Blank pages can also be inserted by declaring a **.page** and **.subpage** container.

4. Anchors

wkgtk-html2pdf offers two indexing modes to generate the PDF sidebar navigation: Classic and Enhanced.

4.1 Classic mode

In Classic mode, the engine automatically scrapes all internal anchors (any `<a>` tag where the href starts with #) and includes them in the PDF index. This is ideal for simple documents where every link is a significant navigation point.

Example:

```
<!-- Automatically indexed in Classic Mode -->
<li><a href="#reference">Reference</a></li>
<p><a href="#section1">Section 1</a></p>
<div><a href="#appendix">Appendix</a></div>
```

All these anchors are valid.

4.2 Enhanced mode

Enhanced mode (`--index enhanced`) is a professional-grade feature that allows for selective indexing. Only elements explicitly marked with the **index-item** class, class will appear in the PDF sidebar. This prevents "index noise" in documents with many internal cross-references.

Example:

```
<!-- This will be included -->
<li class="index-item"><a href="#reference"><span>Reference</span><span>A1</span></a></li>

<!-- This will be ignored -->
<p><a href="#section1">Section 1</a></p>

<!-- This will be included (container type doesn't matter) -->
<div class="index-item"><a href="#appendix">Appendix</a></div>
```

4.3 Sidebar indexing

Most users of technical manuals rely heavily on the PDF sidebar (bookmarks) for navigation. This persistent tree structure is far more practical for digital consultation than a traditional printed index page. Regardless of whether you choose Classic or Enhanced mode, wkgtk-html2pdf automatically constructs a hierarchical sidebar by analysing your HTML structure.

A unique feature of the wkgtk-html2pdf engine is its ability to generate an index from a visible Single Source of Truth. Unlike traditional generators that rely on "header scraping" or hidden metadata declarations, our engine builds the navigation tree directly from the visible numbering system within your document.

By using the section numbers as the primary data point, the hierarchy is nested automatically. This eliminates the need for manual indexing declarations and prevents document "reflow" issues; if the number is visible on the page, it is accurately reflected in the sidebar. This ensures 1:1 parity between the printed table of contents and the digital navigation tree.

4.3.1 Indexing Criteria

For an item to appear in the PDF sidebar index, it must satisfy two conditions:

1. **Unique Linkage** - The source anchor must reference a target with a unique ID (e.g. `<a href=#mylink>`)
2. **Numbered Hierarchy** - The target element must contain a number. To create nested levels (e.g., Section 1.1 inside Section 1.0), use a period separator.

Example:

```
<a href=#setup_guide>Go to Setup</a>
<h2 id=setup_guide>1.1 System Setup</h2>
```

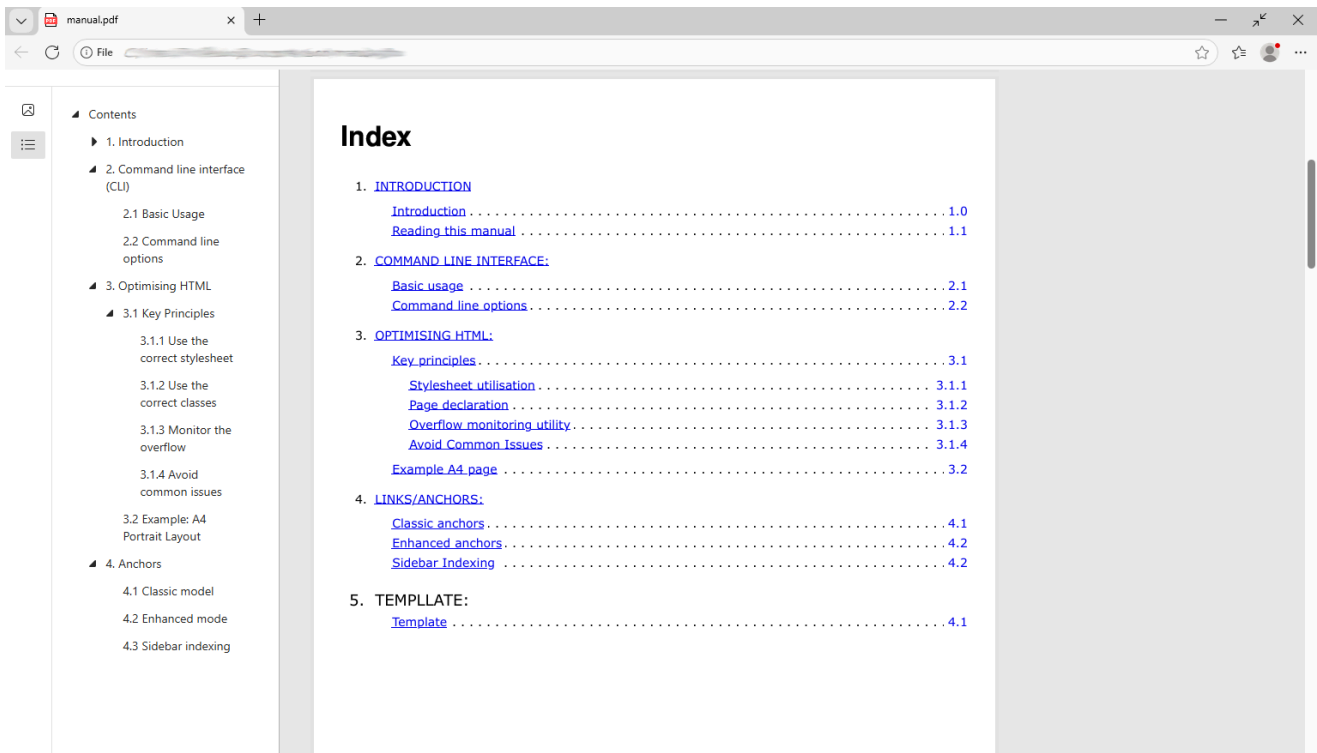
Creating an index item.

The engine will parse "1.1" and automatically nest this bookmark under "1.0" in the PDF sidebar.

4.3.2 Enhanced vs. Classic: The Interactive Hit-Box

The primary functional difference between Classic and Enhanced modes is the management of the "Clickable Area."

In Classic Mode, only the text within an anchor `<a>` is interactive. This often leaves the rest of the line—such as dot leaders or page numbers—inert.



Sample pdf with sidebar index.

The easiest way to determine the difference it to see it by compiling this manual in in both **enhanced** and **classic** mode and click on the links within the index.

Note: This manual is developed to compile in enhanced mode, in classic mode you may notice additional unexpected items in the pdf bookmark bar; this is normal and expected behaviour.

Enhanced Mode solves this by expanding the interactive hit-box to the entire container. By wrapping your anchor in an element with the `.index-item` class, the engine ensures the entire line becomes a clickable link in the final PDF.

To enable enhanced indexing, simply wrap your anchor as follows:

```
<span class="index-item"><a href="#section-target" >4.3.1 Technical Specifications</a></span>
```

Enabling the link in **enhanced** indexing mode.

Why use enhanced mode:

Touch Accuracy - Makes navigation significantly easier on tablets or touch-enabled PDF readers.

Nesting Logic - The engine uses the visible number (e.g., 4.3.1) within the `.index-item` to automatically determine the correct nesting depth (Section 4 -> 4.3 -> 4.3.1) in the sidebar.

Unlike many PDF generators, wkgtk-html2pdf supports redundant indexing. You can place multiple `.index-item` links targeting the same section ID across different pages without "coordinate ghosting." Our engine uniquely maps each physical instance to the target, ensuring that every link remains perfectly interactive regardless of document length.

4.4 Index page

If you have an index page then attaching it to the sidebar is as simple as declaring **toc** within the page element.

Example:

```
<div class="page toc">
  <div class="subpage">
    <!-- Your page content here -->
  </div>
</div>
```

A page declared as the commencement of the table of contents.

The table of contents should be declared exactly once per document.

Note: If it is declared more than once the first declaration that wkgtk-html2pdf finds will be used and the rest will be ignored.

When you declare a Table of Contents/Index page it will be attached to the top level "Contents" bookmark within the index.

5. Artefacts

5.1 Images

wkgtk-html2pdf offers flexible support for image assets. Whether your project relies on remote, local, or embedded resources, the engine ensures high-fidelity reproduction in the final PDF.

Remote URLs - Images served via HTTP/HTTPS work "out of the box." When generating the PDF, the engine fetches and embeds these assets automatically.

Local & Relative Paths - Static and relative paths are fully supported. When using relative paths, ensure the assets are within the working directory and invoke the -r flag (no argument required) on the CLI to grant the engine access to the local file system.

Base64 Encoding For self-contained documents, you may embed images directly into your HTML using Base64 encoding.

If you are viewing the HTML version of this manual, the randomised image following demonstrates a live remote fetch that will be locked into the PDF upon generation.

Note: Remote image is just a static in example for packaging compliance.



Example: A remote asset embedded at runtime.



An image from a static path.

5.2 Remote artefacts

The engine supports most standard remote assets typically declared in the HTML `<head>`, such as external stylesheets and scripts. However, there is one critical exception: the `<base>` element.



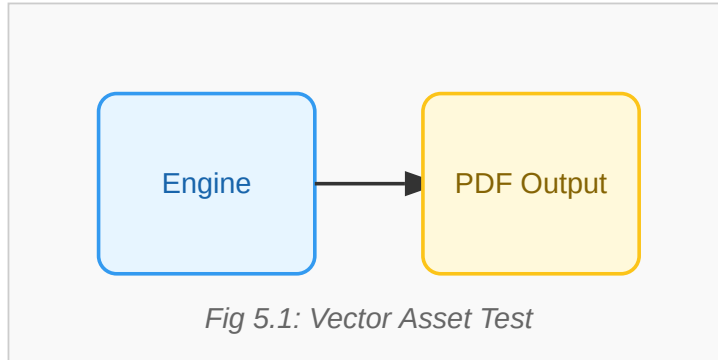
WARNING: Do not use the `<base>` tag. Directing the engine to a base URL will currently break the internal indexing and coordinate mapping. This may be addressed in a future release.

```
<base href="https://www.foo.bar">
```

Do **not** use base URL's.

5.3 SVG (Vector Graphics)

wkgtk-html2pdf is a high-fidelity vector engine. To ensure the smallest possible file size and 100% text searchability, we recommend using Pure Vector SVGs (paths, shapes, and text).



An example vector graphic.



IMPORTANT: Avoid embedding raster images (PNG/JPG) inside SVG files. Our engine prioritises vector integrity; nesting bitmaps within vector containers can trigger full-page rasterisation in the PDF, which disables text selection and impacts render precision.

5.3.1 Layering & Compositing

To maintain 100% text searchability and vector precision when overlaying diagrams on photographs, do not embed the image inside an SVG. Instead, use standard HTML/CSS layering.

Best Practice - Place your background image using an `<img>` tag or CSS `background-image`, then use absolute positioning to overlay your SVG or text elements. This ensures the PDF engine treats the layers independently, keeping your text crisp and your file size lean.

```
<div style="position: relative; width: 400pt; height: 300pt;">
  
  <svg style="position: absolute; top: 0; left: 0;">
    <!-- Vector callouts here -->
  </svg>
</div>
```

Example vector overlaying an image.

6. Page design helper

When converting or generating HTML that works seamlessly as a PDF there are several issues that may lead to unexpected results. While not an exhaustive list the following are most common:

1. **Unexpected extra pages** - It is not unusual for developers to become extremely frustrated at the fact that they have an extra blank page at the end of their document even though they have done everything right.
2. **Overflow** - Content is often missing or going off the page.

This section covers our minimal JavaScript linter that has been developed to detect the most common issues when developing PDF's using HTML.

The JavaScript linter and the styleSheets work together to set up and test the page layout and contents

6.1 CSS Style sheets

When you include the appropriate template for your desired page size and declare a **.page** and **.subpage** <div> element. You will note that you now have something on screen that represents the page size of the PDF you are developing. This will include a margin outlined in blue.

When wkgtk-html2pdf is installed locally copies of some basic style sheets are located in `/usr/share/wk2gtkpdf/` and can be linked thus:

```
/usr/share/wk2gtkpdf/templates
```

Default path of the wkgtk-html2pdf stylesheets.

Note: If you are a designer with no access to the application you can link to or use the templates hosted on our servers instead.

For a more flexible approach you can use the free template maker on our commercial site inplico.uk where you can create dynamic templates with custom page sizes and margins.

6.1.1 Customising Templates

ADVANCED TOPIC (OPTIONAL)

In most cases, if you generate your templates using our free commercial tools then you should not need to manually customise them.

6.1.1.1 CUSTOMISING THE PAGE SIZE

Careful consideration is required when manually defining custom page sizes; your dimensions must result in a whole number of points to prevent the rendering engine from drifting. The high-precision millimetre values used in our templates are calculated specifically to achieve a perfect, integer Point (pt) value based on the 72 DPI (Points Per Inch) standard.

For **US/Imperial** page sizes it is a simple matter of dividing the page size by 72.

$$\text{Points} \times 72 = \text{Inches}$$

For **ISO/Metric** page sizes the calculation is a little more complex.

$$\text{Points} \times \left(\frac{25.4}{72} \right) = \text{mm}$$



WARNING: While MathML is the standard for scientific equations, browser support for "stretchy" operators remains inconsistent. For critical documentation where layout parity is required, we recommend using the CSS Flexbox method provided in our templates to ensure brackets and fractions remain perfectly aligned across all platforms.

In all likelihood, if you are looking to set up a custom page size then you are going to want to convert from mm to points. To do so, for both metric and imperial page sizes is simply a case of flipping the calculation.

US / Imperial:

$$\text{Inches} \times 72 = \text{Points}$$

ISO / Metric:

$$\text{mm} \times \left(\frac{72}{25.4} \right) = \text{Points}$$

Example Walkthrough:

If you require a custom layout of **178mm x 223mm**, you must first find the nearest **whole-point** equivalent to ensure zero-drift rendering:

Width: $178\text{mm} \times (72 / 25.4) \approx 504.56\text{pt} \rightarrow \text{Round to } 505\text{pt}$

Height: $223\text{mm} \times (72 / 25.4) \approx 632.12\text{pt} \rightarrow \text{Round to } 632\text{pt}$

Note: By anchoring your dimensions to an integer point value, you ensure that your internal 12pt/18pt typographic grid remains perfectly aligned with the physical edge of the page, eliminating cumulative rounding errors across multi-page documents.

To ensure 100% parity between your CSS and the hardware, you must "snap" your height calculation to a whole pixel value before assigning it to your page container.

Finally, convert these **integer points** back into the high-precision millimetre values:

Width: $(505 \times 25.4) \div 72 = 178.1528$

Height: $(632 \times 25.4) \div 72 = 222.9556$

```
:root {  
    --page-width: 178.1528;  
    --page-height: 222.9556;  
}
```

Example: Stabilized custom page size.

★ **IMPORTANT:** Always round your calculated Point values to the nearest whole integer before converting back to millimetres for your CSS variables.

6.1.1.2 TYPOGRAPHY/RASTER SYNCHRONISING

While anchoring your layout to Integer Points (pt) ensures typographic consistency and prevents "baseline creep," a secondary calculation is required to satisfy the browser's internal rendering engine (Webkit/Blink/etc.).

Browsers do not paint in millimetres or points; they paint in Physical Pixels (px). At the standard web resolution of 96 DPI, one pixel is exactly 0.75pt. If your calculated page height results in a fractional pixel (e.g., 1190.55pt), the rendering engine will often truncate or "floor" this value to the nearest whole pixel to fit the physical display or print buffer.

Note: If your CSS content height is even 0.001pt taller than the browser's "floored" pixel container, the engine will trigger an Overflow. This is the primary cause of "Ghost Pages"—that dreaded blank page at the end of a document.

$$\text{mm} \times \left(\frac{96}{25.4} \right) \rightarrow \text{Floor(px)} \times 0.75 = \text{Webkit Height (pt)}$$

By using this formula, you are essentially pre-calculating the browser's "Hard Ceiling."

Typographic Logic - Manages the internal baselines so text doesn't drift.

Raster Logic - Manages the external cage so the container doesn't burst.

In our templates this is handled automatically. By rounding down to the nearest pixel, we guarantee that the content area is always exactly equal to, or infinitesimally smaller than, the physical paper, resulting in zero-drift and zero-overflow rendering at any size.

height pure px: (page height * 96 / 25.4);

height floored px: ⌊ height-pure-px ⌋

actual height val: height floored px * 0.75

6.1.1.3 ELEMENT CALCULATIONS

While the Raster Logic manages the outer page constraints, our element-level calculations ensure that every single line of text sits perfectly on a predictable physical pixel grid.

Standard, fluid CSS line-height values (such as 1.4 or 1.5) frequently generate fractional pixel totals (e.g., 16.8px). This forces rendering engines to make subpixel approximations, resulting in unexpected layout anomalies, text clipping, or baseline shifts. The exact same vulnerability applies to point-based font sizes (e.g., 16pt).

To achieve absolute zero-drift rendering, all text containers, font metrics, line boxes, and item margins must be declared in whole pixel values that are mathematically reconcilable with the point unit scale.

The Synchronisation Formula:

Calculate Raw Pixels - Convert the pt value back into pixels.

Snap to Pixel - Floor the result to the nearest whole integer

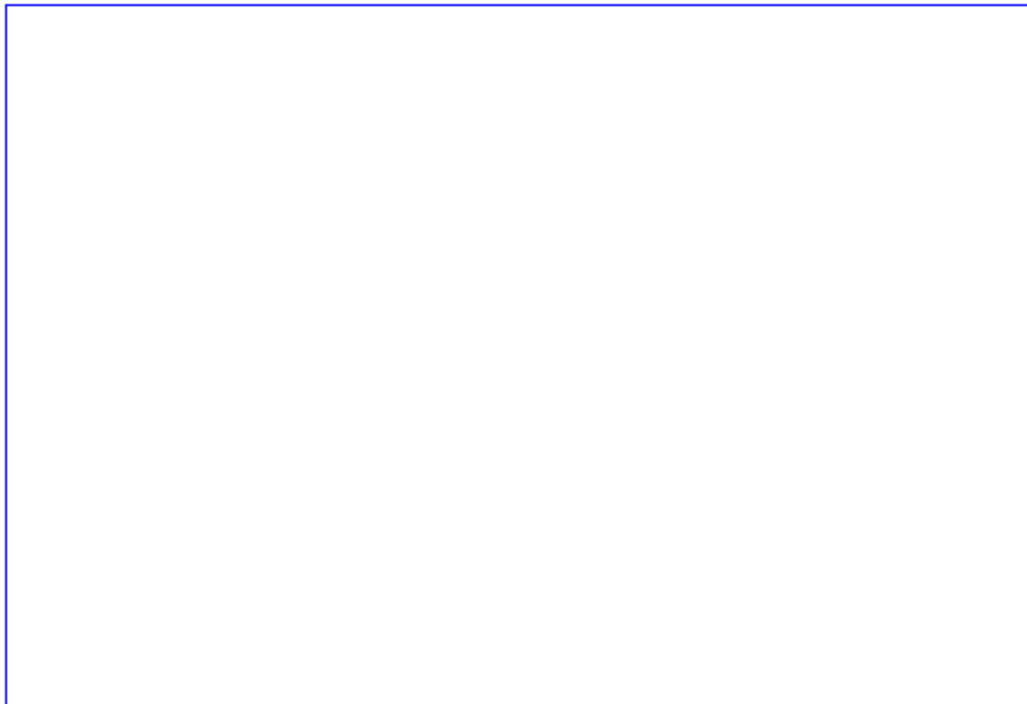
Convert to Points - Convert back to points

Note: Whether it is line height, font size, margins, padding, or any other measurement the formula is the same: Convert it to whole pixels, round it off and convert it back again.

$$\text{pt} = (\lfloor \text{pt} \div 0.75 \rfloor) \times 0.75$$

6.1.2 Using the page templates

Each default template has a margin that is visible to the compositor while they are designing the manual in order to assist with visualisation of the page and to assist in highlighting overflow.



A blank A6 page with margins outlined.

6.1.3 Layout Containers

When designing for print, your choice of CSS layout model (`display`) dictates how reliably content flows across physical page boundaries. While modern browsers handle flex and grid dynamically on screens, print engines require structural predictability.

6.1.3.1 BLOCK LAYOUT (`DISPLAY: BLOCK`)

The standard block layout is the most stable and predictable model for our pixel-locked canvas system.

The Baseline - Because block elements stack vertically using standard flow math, the rendering engine handles coordinate calculations with minimal subpixel drift.

Deterministic Layout - When you apply our integer-pixel snapping formulas to block heights, margins, and padding, the browser maps them reliably to the physical canvas.

Recommendation - Use **`display: block`** as your primary structural wrapper for all text bodies, columns, and general document sections to guarantee zero-overflow predictability.

6.1.3.2 FLEXBOX LAYOUT (`DISPLAY: FLEX`)

Flexbox calculates layouts dynamically based on fractionally available canvas space. In a precision print environment, this automatic sizing introduces severe subpixel rounding risks.

The Risk - Flexbox relies heavily on fractional distribution math. Even when you attempt to force deterministic, floored point/pixel values, the browser's internal engine frequently introduces microscopic fractional remainders (e.g., creating a child height of 45.333px inside a locked 45px space).

The Breakage - These tiny subpixel rounding errors compound rapidly down the DOM tree. Because our system does not allow layout shifting, a fraction-of-a-pixel rounding error can cause an immediate container overflow, resulting in text being unexpectedly cropped at the edge of the .subpage.

The Rule - Use Flexbox with extreme caution. Limit its use to small, single-line horizontal components where dynamic alignment is mandatory (e.g., standardizing fractional brackets/equations as noted in [Section 6.1.1.1](#), inline badges, or icon-text pairs). Never use Flexbox for primary, high-risk vertical structural wrappers.



WARNING: During testing we found that flex box appeared to work for most of the page until it got to the bottom where any element close to the margin boundary would reposition itself randomly.

6.1.3.3 GRID LAYOUT (DISPLAY: GRID)

CSS Grid provides a highly reliable, rigid layout model when you require two-dimensional alignment (such as labels, data tables, or multi-column forms).

Strict Boundaries - Modern browser engines calculate CSS Grid tracks using precise, predictable geometric structures.

Zero-Drift Alignment - Grid items respect their defined constraints and match the integer-pixel grid beautifully, making it an excellent alternative to block layouts.

6.1.3.4 FIELDSET

The semantic HTML `<fieldset>` element exhibits a unique and disruptive box-model calculation bug within web rendering engines when paired with a `<legend>` tag.

The Problem - If the physical padding-top of a `<fieldset>` container is greater than the exact distance between the top of the container's border and the top edge of the `<legend>` text, the engine's subpixel layout math breaks entirely.

The Consequence - The engine miscalculates the external coordinates of the container. This throws out the print alignment, displaces subsequent elements downward, and triggers an artificial box overflow that will cause your content to be cropped at the .subpage boundary.

The Resolution - If you must use a semantic `<fieldset>`, you must normalize and minimize your fieldset and legend spacing in your print stylesheet so the padding never exceeds the legend boundary. If the subpixel math remains unstable, replace the semantic `<fieldset>` with a standard `<div>` styled with a border, and use a standard heading element (`<h3>`) to visually mimic the legend safely.

```
fieldset {  
    padding-top: 16.5pt; /* If this padding exceeds the legend line height, the  
    layout overflows */  
}
```

Avoid padding configurations like this in print stylesheets

6.1.3.5 INLINE CONTAINERS AND SUBPIXEL DRIFT

Standard inline text containers such as `<span>`, `<b>`, and `<i>` are sized dynamically by the browser engine based on vector font metrics, inevitably introducing trailing subpixel rounding variables into the layout canvas.

Under normal scaling conditions, these inline fragments are constrained safely within their parent block containers (such as `<p>` or `<div>`) and do not cause layout anomalies. However, stress testing revealed a highly specific aliasing artifact within the desktop Mozilla Gecko rendering engine. At exactly 150% zoom, subpixel character hinting can expand an inline box boundary outward. If the container is tightly packed, this fractional expansion forces an unexpected horizontal word-wrap, which cascades downward to trigger vertical pagination drift across large multi-page documents (note that at 140%, 160%, or any other zoom scale, the layout remains completely unaffected).

IMPORTANT: This layout artifact is strictly confined to the client-side Gecko desktop engine (e.g. Mozilla Firefox) under a localized zoom configuration. **It does not affect the server-side HTML-to-PDF rendering pipeline in any way.** At the baseline 100% execution scale, the HTML preview and the compiled vector PDF document remain completely identical down to the exact subpixel coordinate line.

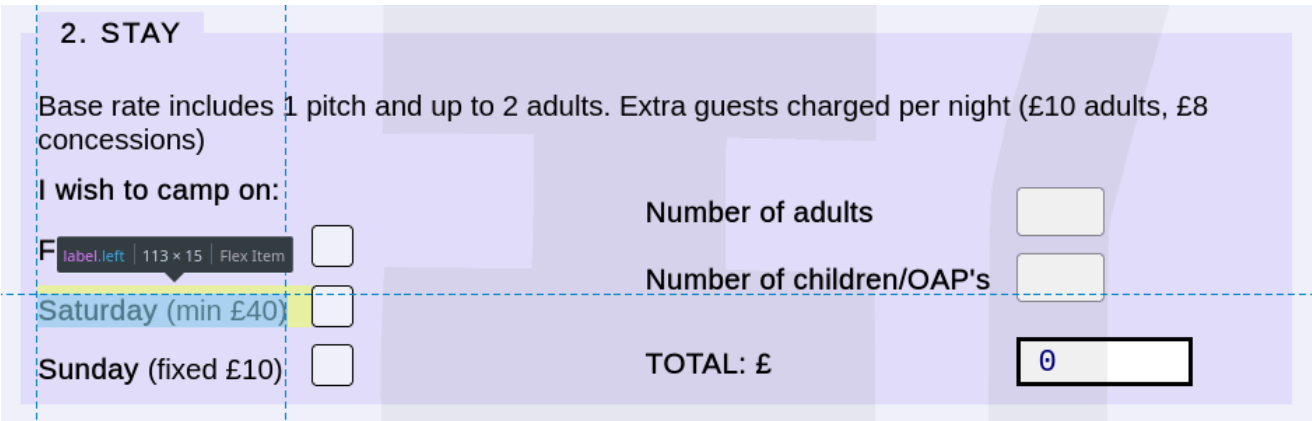


Figure 6.4: The text container rendering normally across standard web browsers.

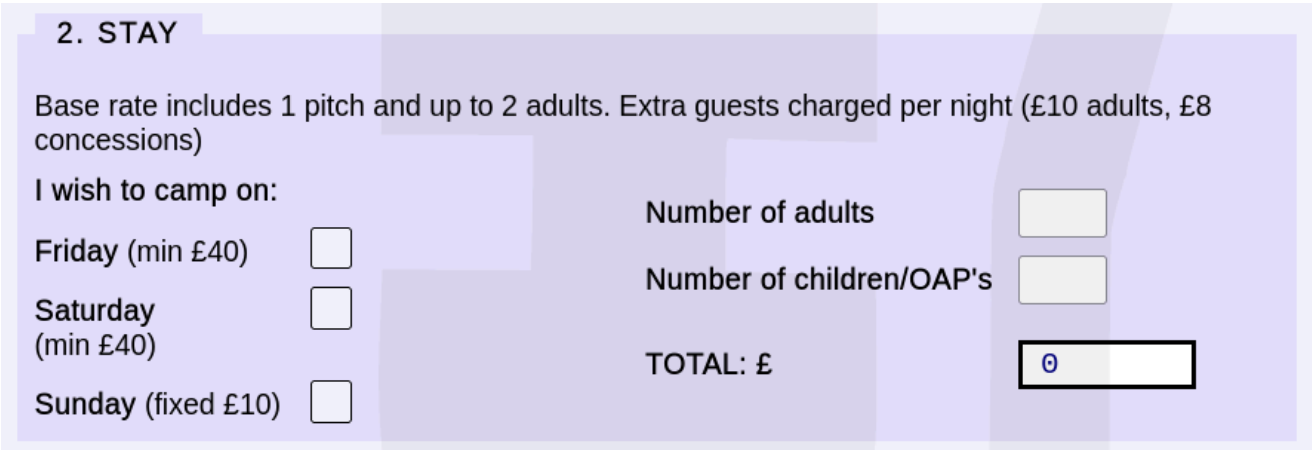


Figure 6.5: Localized line-wrap anomaly tracking inside Firefox at exactly 150% zoom.

6.2 JavaScript helper utility

wk2gtk-html2pdf includes a helper utility to aid with design. It is located in the same folder as the CSS templates and can be used to identify problems before trying to generate a PDF.

There are 2 versions of the linter that can be used depending on how critical your output is.

The basic linter will detect errors that will result in significant drift while the pedantic linter aims to detect most known issues.



WARNING: At the current stage of development and for the foreseeable future the linters should be considered experimental.

To use the utility you need to declare it:

```
<script src=""/usr/share/wk2gtkpdf/overflow-monitor.js"></script>
```

Basic linter declaration

```
<script src=""/usr/share/wk2gtkpdf/overflow-monitor-pedantic.js"></script>
```

Pedantic linter declaration

Once declared, when you open the page in a browser it will automatically monitor changes and notify as and when it detects issues.

6.2.1 Monitored Issues

The specific issues monitored by the design-time linter depend on the audit mode selected. The **Basic Linter** flags critical structural defects that are certain to cause document or rendering failures. The **Pedantic Linter** provides a strict auditing layer, monitoring layout patterns that carry a high probability of subpixel calculation drift.



IMPORTANT: The linter's font validation routine can only determine file availability within your active design-time workspace. It cannot guarantee cross-platform system-wide availability or confirm hosting reachability on external remote servers.

Issue	Basic	Pedantic	Issue
Overflow	✓	✓	Elements exceeding the physical page or grid boundaries.
Missing Fonts	✓	✓	Verification of font availability at design time.
px Declarations		✓	Check for pixel measurement declarations.
em Declarations		✓	Check for relative declarations.
Dirty precision		✓	Check for sub-pixel rounding errors.
Unitless declarations		✓	Detection of unitless line-height scaling multipliers that introduce fractional pixel splits.

★ **IMPORTANT:** Modern browser security models (specifically CORS and local file origin policies) restrict the programmatic DOM access required by the Pedantic Linter to deeply inspect stylesheet rules and font faces. If blocked, an automated alert box will trigger at design time. This restriction is strictly enforced on Linux environments.

Blink Engines (Chromium Architecture)

To bypass local directory isolation boundaries for offline testing on Linux, execute your browser via the command line with file access flags enabled (replace `chromium` with your specific flavor, such as `brave` or `chrome`):

```
chromium --allow-file-access-from-files
```

In strict testing environments with rigorous multi-origin constraints, use this full security override string to safely isolate your layout auditing session:

```
chromium --user-data-dir="/tmp/audit_session" --disable-web-security --allow-file-access-from-files --disable-features=BlockInsecurePrivateNetworkRequests
```

Gecko Engines (Mozilla Firefox)

Extensive testing on Mozilla Firefox reveals that the linter functions out of the box across standard consumer operating systems. The lone exception is Linux-based environments, where strict local origin constraints block DOM stylesheet inspection. To bypass this barrier:

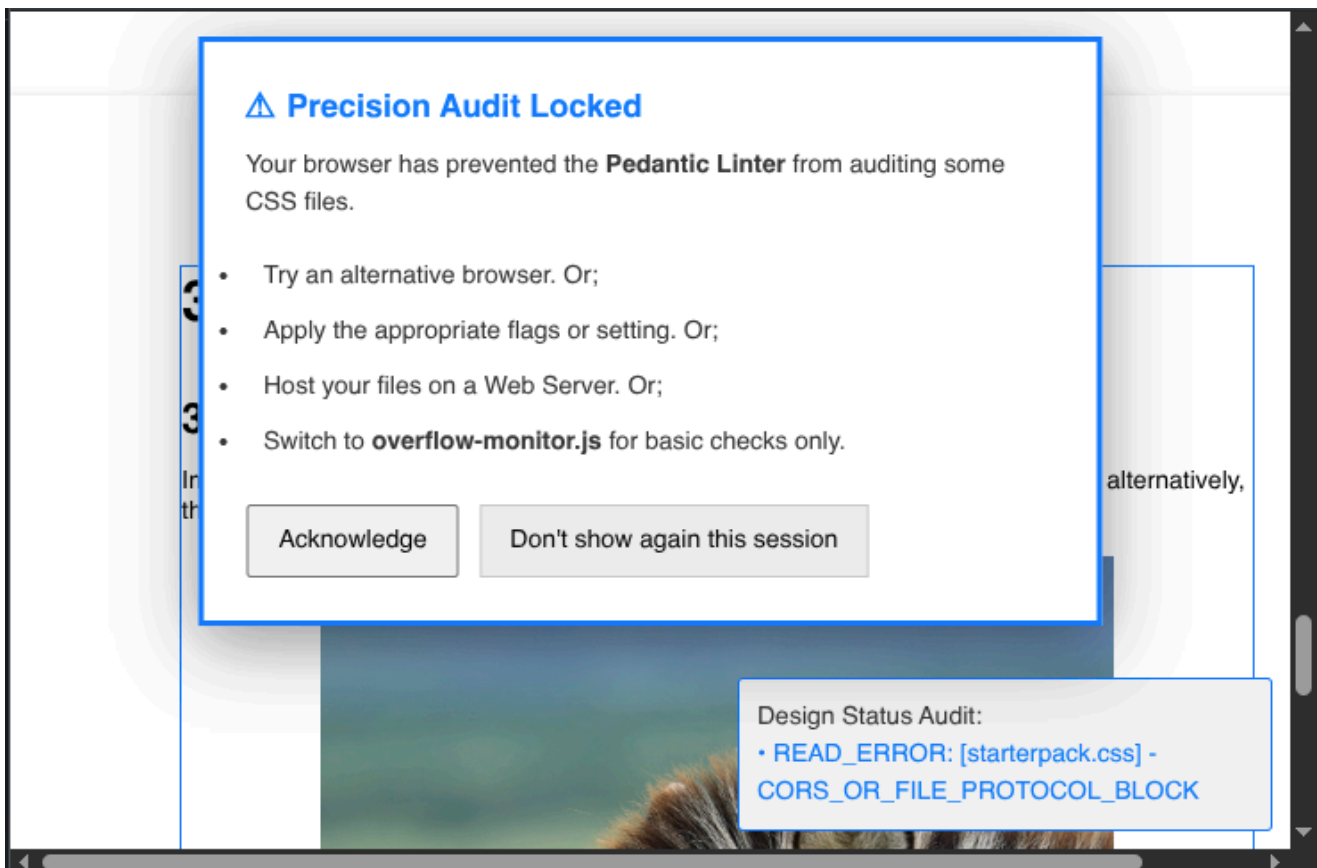
1. Navigate to **about:config** in the address bar.
2. Acknowledge the configuration security warning.
3. Locate the preference key **security.fileuri.strict_origin_policy** and toggle its value to **false**.

WebKit Engines (Linux / Native App Backends)

Despite sharing a core layout lineage with our WebKitGTK production print stack, certain Linux desktop browsers—such as Epiphany (GNOME Web)—enforce an immutable local sandbox model. Because these implementations lack the standard command-line flags necessary to temporarily override origin restrictions, the pedantic design-time linter cannot execute deep local font-metric checks within those specific browser environments. For design-time auditing on Linux, utilizing a configured Chromium or Firefox instance is strictly mandated.

Microsoft Windows — Microsoft Edge and Google Chrome on Windows function natively without requiring custom command-line overrides. If enterprise security policies are tightened or your environment diverges from our standard baseline configuration (running an updated Blink-architecture engine), you may need to pass the same command-line arguments specified for Linux. For standard environments, the system functions seamlessly out of the box.

macOS — Validation testing on macOS remains straightforward. Standard builds of Apple Safari (WebKit), Brave (Blink), and Mozilla Firefox (Gecko) execute the auditing scripts natively. Be advised that older versions of Safari may require enabling the "Allow local file restrictions" flag within the advanced Developer menu if local directory isolation is actively enforced by the operating system.



Notification the linter is disabled by CORS

6.2.2 Font Management

To guarantee that your custom document templates match your screen previews exactly and fit cleanly onto your targeted page counts, we highly recommend utilizing modern, web-optimized open-source fonts rather than legacy desktop fonts.

1. **System-Wide Installation** - Fonts must be installed in the system font directory (e.g., `/usr/share/fonts` on Linux). The rendering library cannot "see" fonts stored only in local user directories or temporary session folders.
2. **Web Fonts (WOFF/WOFF2) (Recommend)** - Modern web font formats are supported, provided they are hosted in a location accessible to the rendering engine at the time of PDF generation.

A Note on Commercial & Proprietary Fonts (e.g., Helvetica)

We advise against specifying proprietary system fonts like Helvetica within your custom template style sheets for two critical reasons:

Platform Licensing Restrictions - Commercial typefaces are strictly licensed by their respective digital foundries. They are not universally installed across web hosting servers or headless PDF generation pipelines, which can lead to unpredictable fallback substitutions.

Vertical Metric Drift - Legacy desktop font files often contain internal spacing tables (hhea and OS/2 line-gap rules) that do not translate cleanly onto digital rendering grids. When forced into tight fractional line heights (such as 9.75pt), these metrics can cause the text layout engine to inject unexpected vertical gaps between wrapped lines or drop entire rows.

For pixel-perfect technical documentation, data forms, and multi-page layouts, our rendering pipeline is optimized for modern typefaces designed specifically for unified cross-platform printing:

To guarantee absolute rendering consistency, protect document layout bounds, and prevent server-side layout failures, it is recommended that you observe a strict font asset protocol.

All custom font assets uploaded or referenced within your templates should be in the WOFF2 (.woff2) format.

Legacy desktop formats such as standard TrueType (.ttf), OpenType (.otf), and legacy PostScript (Type 1) files are not supported by wkhtml2pdf.

6.2.2.1 STRATEGIC CHECKLIST FOR TEMPLATE DESIGNERS

If your templates require any of the following classic typefaces, avoid sourcing legacy desktop files (.ttf or .otf system versions). Their internal structural tables contain conflicting vertical metrics that will cause subpixel line-wrapping drift and massive multi-line text chasms on headless layout engines:

Enforce Clean Multipliers - Ensure your target point sizes scale cleanly into pixel integers when evaluated by rendering engines

Prioritize Metric-Stable Clones - When a layout requires the look of a classic typeface, specify Liberation Sans or Arimo. These options are engineered to deliver identical character spacing without the metric flaws that compromise document stability.

Isolate Structural Elements - (Sometimes necessary) Manage vertical layout spacing with explicit container bounding boxes (padding and margin) rather than relying on font line-height definitions to separate text lines.



WARNING: You should always declare a default global font in your CSS to prevent the browser and the rendering engine from arbitrarily picking one.

If you are generating a PDF on a different machine than the one you are developing on then you must ensure that the relevant fonts are also installed on the generator machine.

If anything, including text, is overflowing the available space the margin will turn red and the notification window will warn of a problem.

When there is a problem with overflow all the margins will turn red however the margins where overflow is present will have a thicker border.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. Nulla consequat massa quis enim. Donec pede justo, fringilla vel, aliquet nec, vulputate eget, arcu. In enim justo, rhoncus ut, imperdiet a, venenatis vitae, justo. Nullam dictum felis eu pede mollis pretium. Integer tincidunt. Cras dapibus. Vivamus elementum semper nisi. Aenean vulputate eleifend tellus. Aenean leo ligula, porttitor eu, consequat vitae, eleifend ac, enim. Aliquam lorem ante, dapibus in, viverra quis, feugiat a, tellus. Phasellus viverra nulla ut metus varius laoreet. Quisque rutrum. Aenean imperdiet. Etiam ultricies nisi vel augue. Curabitur ullamcorper ultricies nisi. Nam eget dui. Etiam rhoncus. Maecenas tempus, tellus eget condimentum rhoncus, sem quam semper libero, sit amet adipiscing sem neque sed ipsum. Nam quam nunc, blandit vel, luctus pulvinar, hendrerit id, lorem. Maecenas nec odio et ante tincidunt tempus. Donec vitae sapien ut libero venenatis faucibus. Nullam quis ante. Etiam sit amet orci eget eros faucibus tincidunt. Duis leo. Sed fringilla mauris sit amet nibh. Donec

Design Issues:

• OVERFLOW

Notification when the content goes outside of the page margin

6.2.2.2 HOW TO RESOLVE OVERFLOWS

If you see the red notification shown above, your content has exceeded the calculated **subpage** height. To fix this, you should:

1. Reduce the amount of text or the size of your images on the affected page.
2. Adjust your **line-height** or **font-size** in your CSS.
3. Check for ghost white space or padding at the bottom of your elements.

In addition to the detection of overflow it should also be capable of detecting missing fonts. This is to be considered advisory and if there are rendering issues the first thing that should be checked is the system wide availability of fonts.

Note: While font detection is highly effective, the technical nature of font substitution means it should be considered advisory rather than an absolute guarantee. If your PDF layout shifts unexpectedly, always verify the system-wide availability of your fonts first.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. Nulla consequat massa quis enim. Donec pede justo, fringilla vel, aliquet nec, vulputate eget, arcu. In enim justo, rhoncus ut, imperdiet a, venenatis vitae, justo. Nullam dictum felis eu pede mollis pretium. Integer tincidunt. Cras dapibus. Vivamus elementum semper nisi. Aenean vulputate eleifend tellus. Aenean leo ligula, porttitor eu, consequat vitae, eleifend ac, enim. Aliquam lorem ante, dapibus in, viverra quis, feugiat a, tellus. Phasellus viverra nulla ut metus varius laoreet. Quisque rutrum. Aenean imperdiet. Etiam ultricies nisi vel augue. Curabitur ullamcorper ultricies nisi. Nam eget dui. Etiam rhoncus. Maecenas

Design Issues:
• FONT_MISSING: dolally

Notification when a missing font is detected

6.2.3 Using Google Fonts & CDNs

Standard web typography services like **Google Fonts** are fully supported. Since these are remote artefacts, WebKitGTK will fetch them at render-time.

Note: While Google Fonts work perfectly in a connected environment, we recommend downloading the `.ttf` or `.woff` files and installing them **system-wide** if you need to generate PDFs in an offline or firewalled environment (such as a secure server or CI/CD pipeline).

7. Hints and Tips

7.1 Page Numbering

While you will note that the pages in this manual are numbered you will also note that they are not referenced and merely exist to give the reader a sense of familiarity; While you are welcome to use page numbers if you so wish, we strongly recommend using Section Numbers (e.g., See Section 6.2) only in your index instead.

Particular care should be taken when porting documents as the likelihood of page references remaining intact are low.

If you do wish to use page numbers for your indexing then we advise that you develop your document in the following order:

1. Prepare the whole HTML document without any **.page** or **.subpage** elements
2. If the index is at the front of the document then insert as many **.page** and **.subpage** elements as would be required to hold the entire index. (skip this step if your index is at the back of the document).
3. Iterate through the document inserting **.page** and **.subpage** elements and closing them off where necessary.
4. Paginate the document applying an **id** to each page number
5. Generate the index in the reserved space

NOTE: It may be necessary to reflow the pagination if you amend the document.

7.1.1 Our Recommendation

In PDF documents it is rare for the page number determined by the reader to be the same as the page number marked at the bottom of the page; this is because PDF readers start counting from the very first page (often the document cover which is not paginated).

1. **Maintenance Zero** - Section numbers are 'set and forget.' They remain accurate regardless of how the content flows across physical pages.
2. **Digital Precision** - Our indexing engine maps the exact XY coordinates of your anchors. Clicking "Section 6.2" in the PDF will always take the reader to the correct content, even if a layout change shifts that content from Page 10 to Page 11.
3. **The 'Reflow' Nightmare** - Referencing physical page numbers in your text requires manual updates every time your template or content changes. Using section-based anchors eliminates this entire category of error.

In this manual, as the page numbers are merely to give the reader a sense of familiarity they are automatically inserted by utilising CSS' built in **counter** function; this is how we do it:

To start the counter from a particular page simply add a class of **.number** to that page

```
<div class="page number">
```

Commence numbering from a given page

Once you have instantiated a **.number** class the following pages will automatically be paginated; it is not necessary to add **.number** to additional pages and will in fact cause the page counter to be reset.



WARNING: Adding the **.number** class to every page will reset the counter to 1.

Here is the CSS for the page numbering used in this manuals

```
div.page.number {
  counter-reset: page 0;
}
div.page.number,
div.page.number ~ div.page {
  counter-increment: page;
}

div.page.number div.subpage::after,
div.page.number ~ div.page div.subpage::after {
  display: block;
  content: " Page - " counter(page);
  position: absolute;
  bottom: 0px;
  right: 0px;
  z-index: 999;
  padding: 2px 8px;
  border-right: 2px solid #23b8e7;
  font-size: 12px;
}
```

An automated page counter positioned towards the bottom right of the subpage

7.2 Remove underlining on PDF

We are all used to seeing links underlined on web pages but for some it can seem an unnecessary distraction when transcribed to a PDF. Albeit subjective (as all design is) personally I like to get rid of the underline but keep the blue colour just to hint to the user that it is clickable.

To change this or any formatting specifically for pdf generation you use the **@media print** directive in your CSS.

```
@media print {
  a {
    text-decoration-line: none !important;
  }
}
```

Remove the underline from links for PDF

7.3 Remove the visible page margin

By default the blue margin used to aid design will be displayed in any HTML version; You should hide it rather than remove it to properly maintain the page dimensions.

```
.subpage {  
    border: solid white !important;  
}
```

Hide the margin guide

7.4 The Reference Baseline Template

To accelerate development, the suite includes `technical-manual.css`. This is a carefully calculated baseline that provides the fundamental geometry for technical documentation.

Features:

Low-Opinion Design - The styles are intentionally generic. We provide the "Plumbing" (vertical rhythm, notification boxes, and index leaders) so you can focus on the content.

The Component Contract - The code snippets provided in the next section (7.5) are mapped 1:1 to the classes defined in this baseline

The template is by no means mandatory, but is the quickest way to get started or to test the suite is suited to your needs is to link to it either from our templates repository or the default folder if you have the library installed (See: [6.1 CSS Style sheets](#))

7.4.1 Provisions

7.4.1.1 NOTE BOX

Note: This note box layout is included in the `technical-manual.css` template.

7.4.1.2 IMPORTANT BOX

★ **IMPORTANT:** This important box is included in the technical-manual.css template.

7.4.1.3 WARNING BOX

 **WARNING:** This warning box is included in the technical-manual.css template.

7.4.1.4 INDEX WITH LEADERS

1. INTRODUCTION

Introduction	1.0
--------------	-----

7.5 Development environment setup

Depending on what we are doing we use both of these setups (sometimes even both at once), but as long as you have something to preview and something to write code on you shouldn't run into difficulties.

7.5.1 Pulsar-edit/VSCode

Without doubt our favourite IDE for developing anything HTML is **Pulsar-edit**.

We are reliably informed that VS Code is another solid alternative that works similarly however we haven't used it ourselves.

On the following pages we have provided the snippets that we use to speed up development in both pulsar edit and VSCode.

 **WARNING:** While we can confirm with some degree of certainty that the pulsar-edit version works we have only conducted limited testing with VSCode.

7.5.1.1 PULSAR EDIT SNIPPETS

```
'text.html.basic':
  'Page and Subpage':
    'prefix': 'hpage'
    'body': """
      <div class="page">
        <div class="subpage">
          $1
        </div>
      </div>
    """
  'Code Box':
    'prefix': 'hcode'
    'body': """
      <div class="code-container">
        <div class="code-container-sub">
          <pre><code>$1</code></pre>
        </div>
      </div>
    """
  'Note':
    'prefix': 'hnote'
    'body': """
      <div class="note-box">
        <span class="note-icon">i</span>
        <div>
          <strong>Note:</strong>
          $1
        </div>
      </div>
    """
  'Warning':
    'prefix': 'hwarn'
    'body': """
      <div class="warning-box">
        <span class="warning-icon">⚠</span>
        <div>
          <strong>WARNING:</strong>
          $1
        </div>
      </div>
    """
```

...cont'd

```

'Important':
  'prefix': 'himportant'
  'body': ""
    <div class="important-box">
      <span class="important-icon">★</span>
      <div>
        <strong>IMPORTANT:</strong>
        $1
      </div>
    </div>
  ""

```

7.5.1.2 VSCODE

```

{
  "Page and Subpage": {
    "prefix": "hpage",
    "body": [
      "<div class=\"page\">",
      "  <div class=\"subpage\">",
      "    $1",
      "  </div>",
      "</div>"
    ],
    "description": "Insert page and subpage container"
  },
  "Code Box": {
    "prefix": "hcode",
    "body": [
      "<div class=\"code-container\">",
      "  <div class=\"code-container-sub\">",
      "    <pre><code>$1</code></pre>",
      "  </div>",
      "</div>"
    ],
  },
},

```

...cont'd

```

    "Note": {
      "prefix": "hnote",
      "body": [
        "<div class=\"note-box\">",
        "  <span class=\"note-icon\">i</span>",
        "  <div>",
        "    <strong>Note:</strong> $1",
        "  </div>",
        "</div>"
      ]
    },
    "Warning": {
      "prefix": "hwarn",
      "body": [
        "<div class=\"warning-box\">",
        "  <span class=\"warning-icon\">△</span>",
        "  <div>",
        "    <strong>WARNING:</strong> $1",
        "  </div>",
        "</div>"
      ]
    },
    "Important": {
      "prefix": "himportant",
      "body": [
        "<div class=\"important-box\">",
        "  <span class=\"important-icon\">★</span>",
        "  <div>",
        "    <strong>IMPORTANT:</strong> $1",
        "  </div>",
        "</div>"
      ]
    }
  }
}

```

★ **IMPORTANT:** When pasting snippets into snippets.cson (Pulsar) or html.json (VS Code), ensure the indentation is flush-left within the body strings. Any leading whitespace in the snippet source will be rendered as a physical offset.

7.5.2 Vim and a browser

If you love vim then you probably don't need us to tell you how to set up your environment but in testing we have found that previewing the output with **Falkon** browser results in the almost real time updates on save without need to manually refresh the page refresh. While probably not the world's greatest browser for daily internet use we have found it the best solution for live previewing while using your favourite text editor (be that vim, nano, or something less hardcore) to design your pages.

8. Calibration

Unlike traditional "Headless" PDF generators that function as black boxes, the wkgtk-html2pdf engine includes a built-in forensic calibration suite. We do not ask you to trust our coordinate mapping; we provide the tools to verify it.

8.1 Why Calibrate

A PDF is a physical map. Even a 0.1pt rounding error in a CSS rendering engine can lead to Point-Creep, where content subtly shifts over a long document. By using the `--calibrate` flag, you generate a 150-page "Stress Test" that proves the engine's 1:1 parity between the virtual DOM and the physical page.

8.1.1 The Dual-Output Strategy

The calibrator generates both a **.pdf** and an **.html** file from a single internal source. This creates a "Digital Reference" that allows for a three-stage validation:

Visual Parity - Open the HTML in a browser and the PDF in a viewer (e.g., Okular or Acrobat). Overlay the windows. The 1mm grid lines should remain surgically aligned across the entire stack.

Repeatability - Compare Page 1 to Page 150. If the 260mm grid line is in the exact same position on both, you have confirmed a zero-leak vertical rhythm.

Physical Truth - Print a calibration page at 100% scale (No "Fit to Page"). Use a steel ruler to verify that 100mm on the paper is exactly 100mm in the real world.

8.1.2 When to Calibrate

Because the engine relies on the host system's WebKit and font libraries, we recommend a calibration pass during the following "High-Stakes" events:



WARNING: Always re-calibrate after a system-wide update. Updates to `webkit2gtk` or `pango` can subtly alter font metrics and baseline calculations.

Initial Deployment - Establish a "Forensic Baseline" for your server environment.

Browser Updates - If your regular browser updates or you change your regular browser you should always calibrate it to the engine.

Template Revision - If you are experiencing drift or unexpected results run the calibration to verify the integrity of the engine.

Audit Verification - Include a datestamped Calibration Certificate (the footer of the grid) in your project's archive to prove the manual was rendered on a validated engine.

Contractual Accountability - If you outsource document design to third parties, the calibrator provides a "Clean Room" proof of the engine's integrity. If a designer's layout drifts or overflows, the 150-page calibration grid serves as forensic evidence that the fault lies in the CSS Architecture, not the Rendering Engine. It eliminates the "He said, She said" disputes by providing a mathematically perfect baseline.

The Calibrator is not just a technical test; it is a Contractual Safeguard. Always run a calibration pass before signing off on externally developed templates to ensure they adhere to the physical geometry of the 'Cage' Standard.

8.2 How to Calibrate

To ensure the engine is performing as it should you first need to generate the test documents using the command line interface.

`wkgtk-html2pdf` will generate a US ANSI A or ISO A4 standard page test document depending on the given command line argument; Both documents will have the same IEEE margins.

```
wkgtk-html2pdf --calibrate US
```

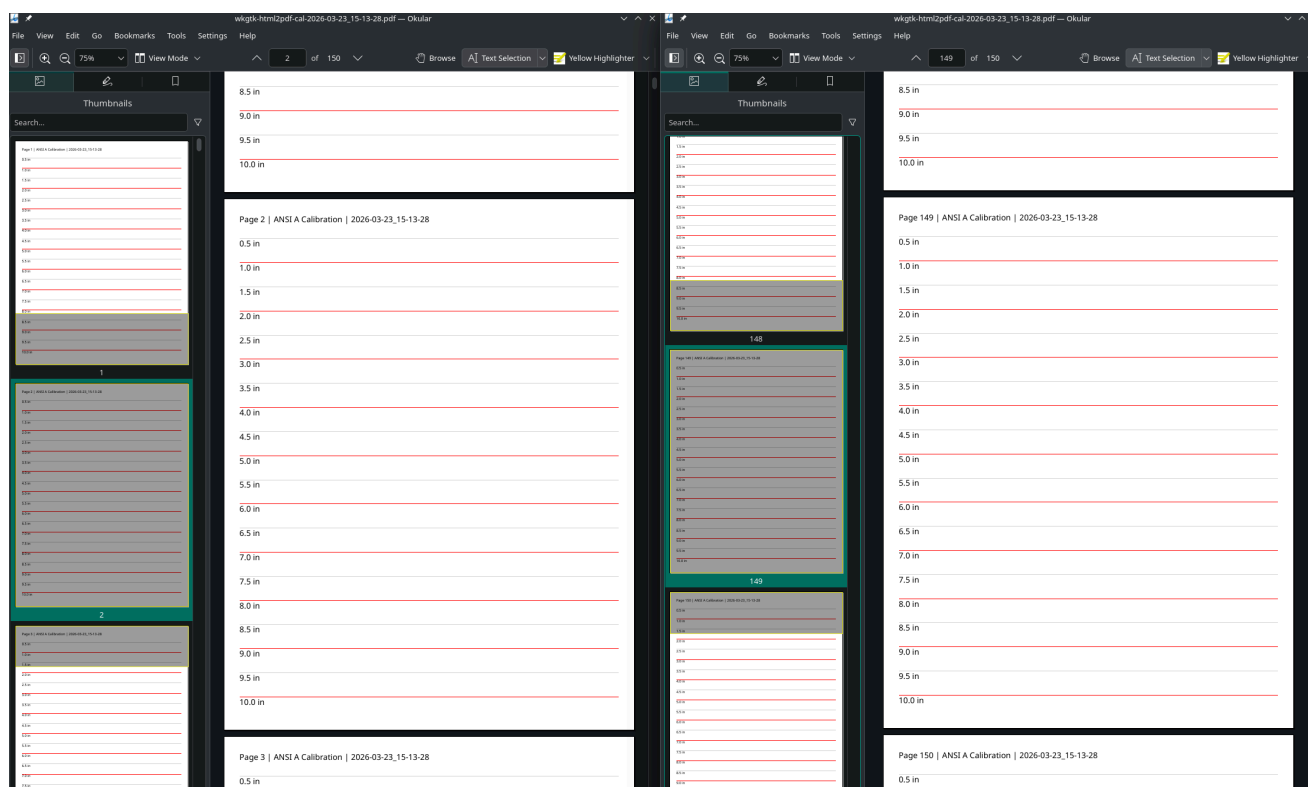
Example to generate a 150 page ANSI A test document

Note: To generate an ISO A4 test document use `--calibrate ISO` instead.

Upon completion 2 files will be generated; these files will be located in the folder where you ran the command and will be named **wkgtk-html2pdf-cal-** appended with a timestamp and the appropriate extension **.pdf** or **.html**

8.2.1 Creep Validation

HTML doesn't actually generate pages at all so the engine is entirely reliant on coordinates to construct a PDF from an HTML layout. If these coordinates are incorrect (even by as little as a pixel) this will result in an accumulative creep problem where whole lines will be cropped appear on the wrong page. To ensure the engine isn't causing this look at the first page and the last page of the test document and they should be identical.



Creep validation - All pages are identical.

Note: For the highest degree of certainty use the ISO version for this test as it has more lines

8.2.2 Testing Browser Calibration

★ **IMPORTANT:** If ever you have any appearance issues this is the place to start.

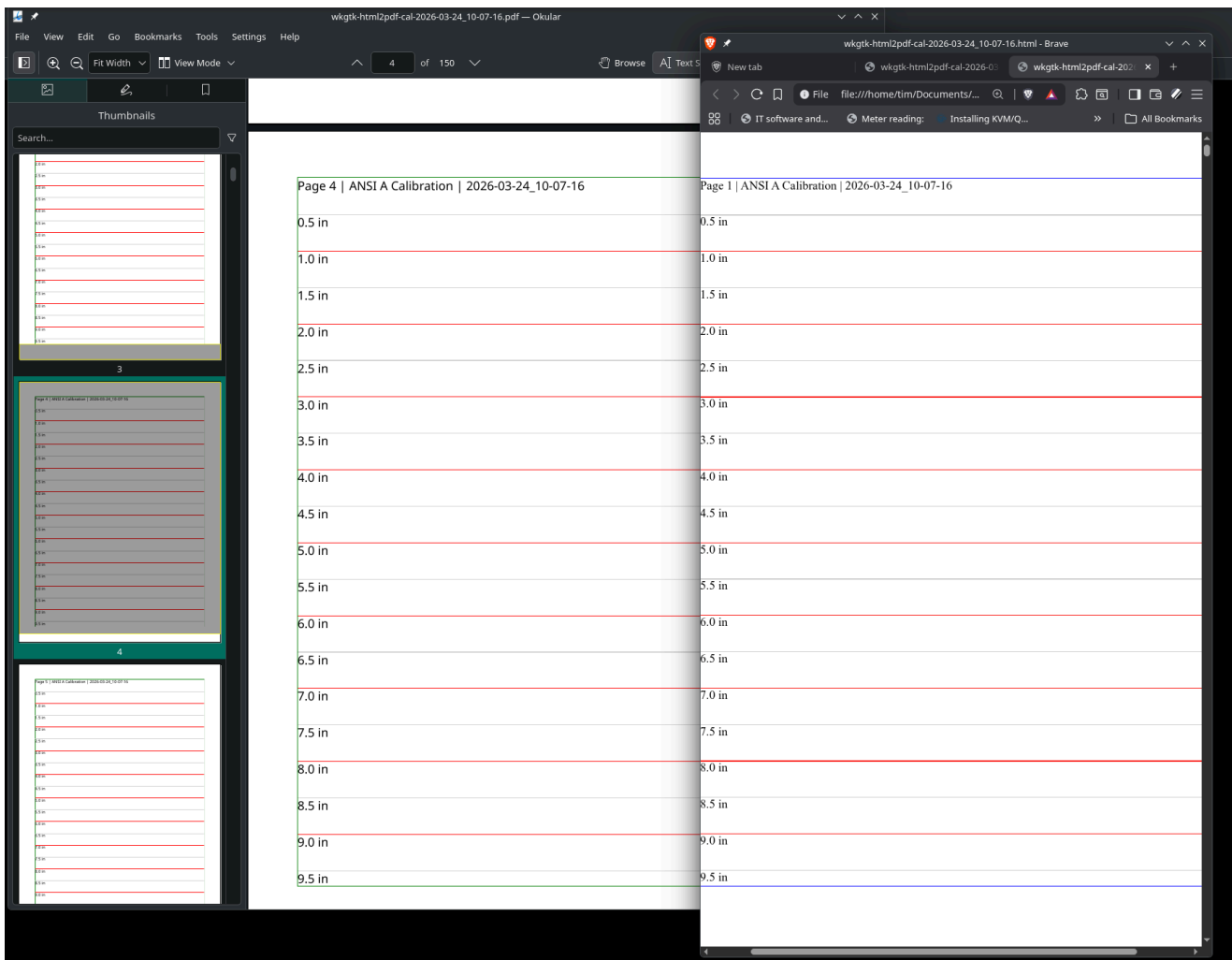
During development of the library, the command line interface and the templates we tested browser from **Falkon** on KDE to **Chrome** on Microsoft Windows and managed to ensure parity throughout the spectrum however we have no control over the future development of browsers and therefor highly recommend that you calibrate your browser regularly.

i **Note:** We find it easier on the eyes to use the US calibration page for this test as it has fewer rule lines.

To conduct the test

1. **generate the calibration documents** - and open them up in a PDF reader (eg. Adobe Acrobat or Okular) and a browser of your choice.
2. **Set up the browser** - so you can see a single entire page on your screen (A border-less browser is best for this).
3. **Align the PDF** - Set your PDF viewer to scale to width and with a full page viewable drag the sides to scale the page until the upper and lower green margins in the PDF align with the upper and lower blue margins of the page in the browser.
4. **Check** - the lines in between; If they line up then the browser and the PDF engine are calibrated and therefore provided you follow the guidance set out in this document your documentation will align also.

i **Tip:** During testing we discovered that the least forgiving browser was **Falkon**. This meant that if we made a mistake with our page design or our style sheets it stood out. For the purposes of development we don't consider this a flaw, but an asset and therefore we recommend **Falkon** if you have the opportunity to use it; although any browser will work, some are more forgiving. At the time of testing Falkon still used an older version of Blink so we expect by the next release that there will be some parity with more recent versions of Chrome and that will likely lead to the loss of the advantage.



Testing browser calibration.

Tip: Setting the PDF scaling to 88.4% while leaving the browser on 100% should give you a pretty good match on most systems.

8.2.3 Printer Calibration

Testing printer calibration is simply a matter of printing a page out and measuring it. The margins for both US and ISO page sizes have been explicitly set to:

Top/bottom margins - 15.8mm

Left/Right margins - 16.9mm



WARNING: Switch off Page Scaling/Scale to fit before testing.

Most printers have a tolerance level set by the manufacturers, please be aware that the best that can be expected is within that tolerance.

8.3 Quality Control

To ensure that the resultant PDF is an accurate representation of the HTML and to aid in troubleshooting we recommend these guideline are followed:

8.3.1 Structural Integrity

- [] **Use the pedantic linter** to catch the majority of errors likely to cause drift.
- [] **Line height** - should be explicitly set in **pt** values rather than implicit multipliers
- [] **No missing fonts** - Not only must fonts be verified at design time, they must also be installed system wide on the machine producing the PDF.

★ **IMPORTANT:** $\text{line-height} \approx \lfloor ((\text{multiplier} \times \text{font-size}) \div 0.75) \rfloor \times 0.75$

8.3.2 Engine Integrity

- [] **Browser Test Pass** - Browser calibration has been tested.
- [] **Drift Test Pass** - No drift can be detected on the test PDF document between the first and the last page.

8.3.3 Index Integrity

- [] **Target Verification** - Each bookmark in the PDF takes you to the correct destination.
- [] **Nesting** - Each link is nested within the correct parent.
- [] **All Links Present** - Every expected link is contained within the bookmark bar
- [] **No Extra Links** - There are no extra links within the bookmark bar.
- [] **Content page has been set** - Clicking **Contents** takes you to the index.

9. C++ API

The wkgtk-html2pdf library provides a professional-grade C++ interface designed for high-performance document automation. Starting with Version 1.0.0, the API utilizes the Pimpl (Pointer to Implementation) idiom, ensuring a stable Application Binary Interface (ABI). This allows the internal rendering engine to be updated without requiring the host application to be recompiled.

9.1 Key Concepts

The wkgtk-html2pdf C++ API is designed for professional document automation where long-term binary compatibility is a requirement. To achieve this, the library adheres to two core principles:

9.1.1 The Binary Wall (Pimpl)

Every public-facing class such as PDFprinter and html_tree utilises the Pointer to Implementation (Pimpl) idiom. The header files contain no private members from third-party libraries (WebKit, PoDoFo, or GLib).

Benefit: You can upgrade the underlying rendering engine by simply replacing the shared object (.so) file. Your host application does not require recompilation

9.1.2 Thread Safety & Initialization

For non-GTK host applications, the rendering engine utilizes a **Thread-Safe Singleton** pattern to manage the underlying GTK environment.

```
icGTK::init();
```

Initializing the Global GTK Context.

This method must be invoked before any PDF generation calls are executed. It initializes the heavy WebKit infrastructure within a **dedicated worker thread**, isolating the GTK initialisation from your application's main execution loop. As a "Set-and-Forget" object, it provides no public getters or state manipulators; its sole purpose is to maintain the lifecycle of the rendering thread.



WARNING: Do not invoke **icGTK::init()** from within an existing GTK-based process. Initializing a second global context is redundant and will lead to an immediate application crash.

9.2 The `html_tree` Class (DOM Construction)

The `html_tree` class allows for programmatic construction of well-formed HTML without manual string concatenation. `cpp`

```
#include <wk2gtkpdf/ichtmltopdf++.h>

// 1. Create the Root
phtml::html_tree dom("html");

// 2. Nest Elements
phtml::html_tree *body = dom.new_node("body");
phtml::html_tree *h1   = body->new_node("h1");

// 3. Set Content (ABI-safe strings)
h1->set_node_content("Professional PDF Report");

// 4. Extract HTML string
phtml::process_nodes(&dom);
const char *html = dom.get_html();

// 5. Cleanup
phtml::PDF_FreeHTML(html);
```

Minimal example using built in DOM constructor.

9.3 The `PDFprinter` Class (Rendering)

The `PDFprinter` class manages the conversion of HTML strings into PDF documents or raw binary data.


```
#include <wk2gtkpdf/ichtmltopdf++.h>

// Initialize the Global GTK Loop (Call once)
icGTK::init();

phtml::PDFprinter pdf;

// Set input HTML and Output filename
pdf.set_param(html, "report.pdf");

// Set Layout parameters
pdf.layout("A4", "portrait");

// Execute Rendering
pdf.make_pdf();
```

Generating a PDF.

9.4 Working with BLOBs

For advanced workflows (e.g., post-processing with ImageMagick), the library can return the PDF as a raw memory buffer.

```
pdf.set_param(html); // No filename provided = BLOB mode
pdf.make_pdf();

// Retrieve Binary Data
phtml::PDF_Blob binDat = pdf.get_blob();

// ... process binDat.data (size: binDat.size) ...

// Explicitly free the library-allocated memory
phtml::PDF_FreeBlob(binDat);
```

9.5 Variadic Helpers

To facilitate dynamic content generation, the API includes printf-style helpers that handle internal buffering automatically.

```
new_node_f(const char *format, ...)
set_node_content_f(const char *format, ...)
```

9.6 Indexing Configuration

The wkgtk-html2pdf engine provides three distinct modes for generating PDF sidebar bookmarks (the document outline). This is controlled via the `phtml::index_mode` enumeration.

9.6.1 The `index_mode` Enum

When calling the rendering methods, you must pass one of the following flags:

Constant	Description
<code>index_mode::OFF</code>	Default. No sidebar bookmarks are generated.
<code>iindex_mode::CLASSIC</code>	Scrapes all <code><a></code> tags with internal <code>#</code> anchors. See 4.1 Classic mode
<code>index_mode::ENHANCED</code>	Only indexes elements explicitly marked with the <code>index-item</code> class. See 4.2 Enhanced mode

9.6.2 API Implementation

To enable indexing within your C++ application, pass the mode as the third argument to the parameter setup function.

```
#include <wk2gtkpdf/ichtmltopdf++.h>

// 1. Initialise the printer
phtml::PDFprinter pdf("file:///path/to/resources/");

// 2. Set parameters with Indexing Mode
// Signature: set_param_from_file(const char* infile, const char* outfile,
// index_mode mode)
phtml::index_mode myMode = phtml::index_mode::ENHANCED;

pdf.set_param_from_file("input.html", "output.pdf", myMode);

// 3. Configure layout and render
pdf.layout("A4", "portrait");
pdf.make_pdf();
```

9.6.3 Strategic Usage

Off - For documents that do not have `.page` and `.subpage` classes the only option is to leave indexing off.

Classic Mode - Best for legacy HTML documents where headers are already wrapped in standard anchors. It requires zero modification to the HTML but can result in a "noisy" sidebar if the document contains many cross-references.

Enhanced Mode- The professional choice for complex manuals. By requiring the `.index-item` class, it allows the developer to precisely curate the navigation tree and, as noted in [Section 4.3.2](#), expands the clickable hit-box to the entire container for better UX in PDF viewers.



WARNING: Setting indexing to enhanced mode without using the bundled templates and declaring `.page` and `.subpage` `<div>` containers will cause undefined behaviour.

9.7 GTK Development

Because a single Linux process cannot safely load both `libgtk-3` and `libgtk-4` simultaneously, `wkgtk-html2pdf` is distributed in two distinct builds. Selecting the correct version depends entirely on your application's environment.

9.7.1 Headless applications

The library supports both GTK3 (WebKit 4.1) and GTK4 (WebKit 6.0) targets. Thanks to our Pimpl-shielded architecture, the headers and public API symbols remain identical across both versions. To switch your build target, simply update your `pkg-config` flags:

For GTK3 - `pkg-config --libs wk2gtkpdf-4`

For GTK4 - `pkg-config --libs wk2gtkpdf-6`

9.7.2 GTK Applications

Attempting to link a GTK3-based host application against the GTK4 version of the library (or vice-versa) will result in an immediate Segmentation Fault or a Critical Gdk-Error at startup. This is not a limitation of the library, but a fundamental restriction of

the GTK display backend, which cannot coexist with multiple global versions in a single process space.

Ensure that your build target matches your host environment as described in [Section 9.7.1](#).

9.7.3 Headless Daemons

As WebKitGTK requires a valid display surface to initialize the rendering pipeline, libwk2gtkpdf is bundled with a dedicated background service. This service manages a virtual framebuffer (Xvfb), providing a "Ghost Display" when no physical hardware is available.

Note: During development, we attempted to generate PDFs using the Weston (Wayland) compositor as a headless alternative to Xvfb. However, while Weston is suitable for general rendering, it did not support stable PDF generation at the time of publication. We will continue to monitor Wayland's headless maturity; until then, Xvfb remains the mandatory standard for server-side production.

For high-availability services (such as automated statement delivery), the engine must be decoupled from the main application thread. This ensures that the "heavy lifting" of WebKit rendering does not block your application's supervisory logic or network listeners.

The recommended lifecycle for a long-running service involves initializing the global singleton in your main() entry point and delegating the document production to a dedicated worker thread.

```
int main(int argc, char** argv) {
// 1. Initialize the Global GTK Context (Once per lifetime)
icGTK::init();
// 2. Spawn the Industrial Delivery Thread
std::thread t1(& {run_statement_delivery_service();});
t1.join();
return 0;
}
```

This pattern has been forensically verified for stability. During a 20-day "Longevity Audit" of the Inpicare Automated Account Statement service, this exact implementation demonstrated:

Zero-Leak Throughput - Continuous generation of PDF statements without cumulative memory growth.

Internal Library Compatibility - The `make_pdf()` call remained stable even when nested three levels deep within external shared libraries (.so).

Thread Safety - No collisions between the WebKit rendering thread and the host application's timeout management logic.

★ **IMPORTANT:** The `icGTK::init()` call is a Global Anchor. Once set in `main()`, any thread within the same process space can safely trigger a PDF render, provided the host is built against the correct GTK target (See [Section 9.7.1](#))

9.7.3.1 SERVICE DEVELOPMENT

The most critical failure point for a headless service is the termination of the Xvfb display server. Because WebKitGTK maintains a persistent connection to the display surface, the loss of Xvfb will cause an immediate, unrecoverable crash of your host application.

`libwk2gtkpdf` communicates directly with the `systemd-logind` and D-Bus bus. If the mandatory `xvfb.service` is inactive, the engine will:

Probe - Check the `ActiveState` of the virtual framebuffer via `org.freedesktop.systemd1`.

Resuscitate - Issue a `StartUnit` command directly to the Systemd Manager if the display is missing.

Wait - Synchronize the rendering thread until the virtual display surface is confirmed "active".

i **Note:** There is no need to add `xvfb.service` to your daemon's configuration; Your daemon starts, initialises `libwk2gtkpdf`, `libwk2gtkpdf` then starts `xvfb` if it isn't already running.

```

[Unit]
Description=Foo Daemon
# Ensure the network is up so we can fetch assets or talk to APIs
After=network-online.target
Wants=network-online.target

# Prevent the service from infinite-looping if it fails
StartLimitIntervalSec=60s
StartLimitBurst=5

[Service]
# self-healing policy
Restart=on-failure
RestartSec=5s

# The user/group should have write access to your logs and temp folders
User=bar
Group=bar

Type=simple
ExecStart=/usr/bin/your-daemon-name --verbose --config /etc/foo/config.json

# Clean shutdown handling
KillMode=process
ExecReload=/bin/kill -HUP $MAINPID

[Install]
# This ensures the daemon starts automatically at boot
WantedBy=multi-user.target

```

Example service file.

To prevent the engine from waiting for desktop-only services, `init()` explicitly neuters the following background portals:

Portal Deactivation - `GTK_USE_PORTAL=0` ensures the engine doesn't try to use Flatpak-style "Save As" dialogues.

Bridge Suppression - `NO_AT_BRIDGE=1` prevents the engine from looking for Accessibility tools that aren't present in headless mode.

VFS Lock - `GIO_USE_VFS=local` forces WebKit to use the local filesystem, bypassing network-mount delays.